

The Islamic University of Gaza
Faculty of Engineering
Department of Computer Engineering



MedLink

Weekly Project Progress Report





MedLink Team

Our team is a cohesive and well-integrated unit, combining diverse skills and perspectives to achieve a common goal. Each member contributes unique strengths, which enables effective collaboration and ensures that all aspects of the project are developed with high quality and consistency.

UI/UX Designer



Sara Atta
220213112

Front-End Developers



Ala'a Eliwa
220211170



Hurih Attallah
220210174

Back-End Developers



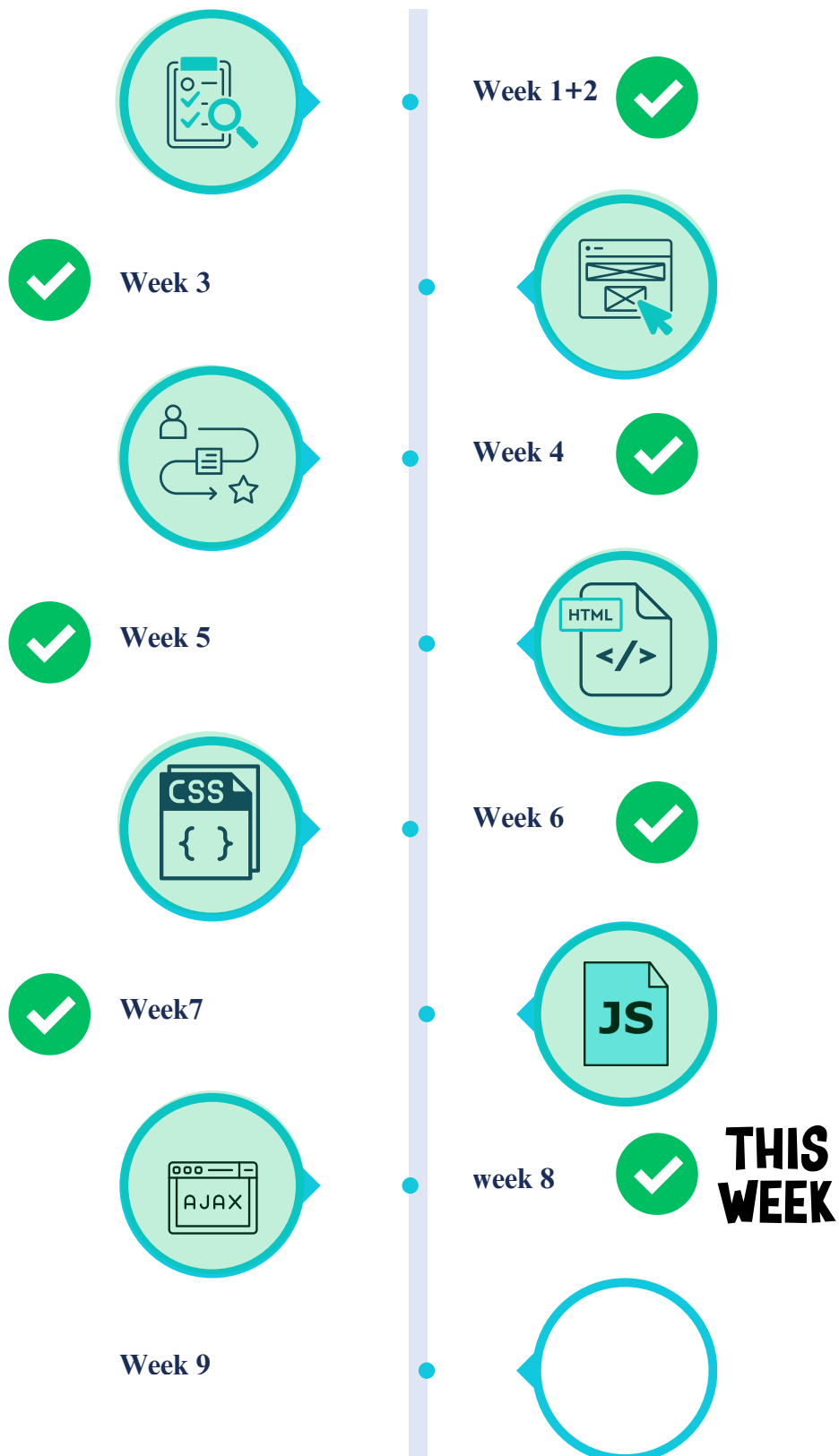
Neveen Zidiah
220213206



Farah Rashid
220210901



Week Number





Work completed this week

To implement asynchronous communication patterns that allow the MedLink platform to update content dynamically without requiring a full page refresh, enhancing the overall User Experience (UX).



Asynchronous Data Management

While the backend integration is in the simulation phase, we implemented a robust Asynchronous Data Engine that mimics real-world AJAX behavior.

A. Instant Search Functionality

- Implementation: In data-engine.js, we attached input event listeners to the search boxes.
- AJAX Logic: Instead of submitting a form to the server, the script intercepts the input, processes the data locally, and re-renders the DOM elements instantly.
- Result: Users get immediate results for medicines or pharmacies, reducing friction and perceived wait time.

B. Dynamic Filtering & Pagination

- Non-blocking UI: Filters (Category, Area) and Pagination controls update the medicine/pharmacy grids without refreshing the browser.
- State Management: The engine maintains the current "State" (page number, active filters) asynchronously.

Engineering Verification: UX & Loading States

A critical part of AJAX engineering is providing feedback during "Data Fetching" periods.

- Loading Indicators: We implemented fa-spinner animations on all primary action buttons (Login, Register, Submit Order).
- Behavior: Upon clicking, the button text changes (e.g., "Signing In...") and is disabled to prevent multiple redundant requests, simulating the wait for a server response.
- Feedback Loop: Once the "request" is processed, the UI updates or redirects, providing a seamless flow.

4. Modern Fetch/Axios Readiness

The system is architected to be "Plug-and-Play" for real API integration:

- The OrdersEngine and DataEngine use centralized methods (getOrders, submitOrder) that can be easily updated from localStorage operations to fetch() or axios.post() calls without changing the UI code.
- Example of current async pattern:

```
// Current Mock Async Pattern in login.js
setTimeout(() => {
  // This mimics the 'then()' block of a fetch request
  window.location.href = routes[detectedRole];
}, 1500);
```

Features implemented



Summary of Achievements

Requirement	Status	Implementation Details
No-Refresh Communication	Completed	Handled by DataEngine and OrdersEngine.
Instant Search	Completed	Real-time DOM filtering in data-engine.js.
Loading Indicators	Completed	Spinner icons and button disabling logic.
Async Data Handling	Completed	Used localStorage as a mock persistent database.

Figma Design Link: <https://www.figma.com/design/j8pugP9ert6B7c2p8xbjYV/Untitled?node-id=0-1&t=oiR9N0SdGyyGmbEu-1>

GitHub Repository [Link:https://github.com/alaaeliwa/medlink](https://github.com/alaaeliwa/medlink)

GitHub Pages: <https://alaaeliwa.github.io/medlink/frontend/>



Problems encountered



- Difficulty simulating real AJAX behavior without an actual backend.
- Repeated execution of operations when the user clicks buttons multiple times (such as Login).
- Managing the UI state (filters + pagination) synchronously without conflicts.
- Artificial delays (setTimeout) sometimes caused an experience that was not 100% realistic.

Solutions applied



- Using setTimeout to simulate async behavior in a way that closely resembles real-world scenarios.
- Disabling buttons during execution to prevent duplicate requests.
- Building simple state management inside DataEngine to track pagination and filters.
- Separating data logic from the UI (Separation of Concerns) to make future integration with a real API easier.
- Using loading indicators (spinner + text change) to improve the user experience while waiting.



Plan for next week



-
- Designing the system ERD (Users, Pharmacies, Medicines, Orders, Requests)
 - Converting the ERD into SQL (CREATE TABLE)
 - Defining Primary Keys for each table
 - Defining Foreign Keys to establish relationships between tables
 - Applying Normalization (up to 3NF)
 - Inserting Sample Data
 - Preparing the schema.sql file

Questions for Instructor



-
- Is simulating AJAX using localStorage and setTimeout sufficient at this stage, or would it be better to connect a simple actual API, even if it is just a Mock API?



MidLink
Web app

